

Byte-Pair Encoding (BPE) Tokenization

Byte-Pair Encoding (BPE) প্রথমে তৈরি হয়েছিল টেক্সট কমপ্রেস করার অ্যালগরিদম হিসেবে। পরে **OpenAI** এটা ব্যবহার করে **GPT model pretraining**-এর **tokenization**-এ। এখন অনেক Transformer model-এ BPE ব্যবহৃত হয়, যেমন:

- GPT
- GPT-2
- RoBERTa
- BART
- DeBERTa

সহজভাবে বললে, **BPE** বড় শব্দকে ছোট ছোট অংশে ভেঙে **token** বানায়, যাতে model শব্দগুলো আরও কার্যকরভাবে শিখতে পারে।

BPE আসলে কী করে?

ধরো, একটা model-এর কাছে পুরো একটা শব্দ না রেখে, শব্দের ছোট ছোট অংশ রাখা হলো।

যেমন:

- hugs → hug + s
- playing → play + ing

এভাবে model নতুন শব্দও বুঝতে পারে, কারণ সে শব্দের অংশগুলো আগে থেকেই শিখে রাখে।

১) Training Algorithm — BPE কীভাবে শেখে?

BPE training শুরু হয় corpus (মানে বড় টেক্সট ডেটা) থেকে সব **unique word** বের করে।

ধরি, আমাদের corpus-এ এই ৫টা শব্দ আছে:

"hug", "pug", "pun", "bun", "hugs"

এখন এই শব্দগুলো লিখতে যে অক্ষরগুলো লাগছে, সেগুলো দিয়ে প্রথম vocabulary বানানো হবে।

সেটা হবে:

["b", "g", "h", "n", "p", "s", "u"]

মানে শুরুতে vocabulary-তে শুধু এক অক্ষরের **token** থাকবে।

বাস্তবে কী হয়?

বাস্তব ক্ষেত্রে base vocabulary-তে সাধারণত থাকে:

- সব ASCII character
- অনেক Unicode character-ও

যদি tokenization করার সময় এমন কোনো character আসে যেটা training corpus-এ ছিল না, তাহলে সেটা **unknown token** ([UNK]) হয়ে যায়।

এ কারণেই অনেক NLP model **emoji** বা অদ্রুত character ভালোভাবে handle করতে পারে না।

GPT-2 / RoBERTa-এর বুদ্ধিমান সমাধান

GPT-2 এবং RoBERTa একটা clever trick ব্যবহার করে।

ওরা character দিয়ে না দেখে **byte** দিয়ে শব্দ দেখে।

এতে:

- vocabulary size ছোট থাকে (মোট **256 byte**)
- কিন্তু প্রায় সব character represent করা যায়
- unknown token হওয়ার ঝামেলা কমে যায়

এটাকে বলে:

Byte-level BPE

২) Merge Rule কী?

শুরুর vocabulary পাওয়ার পর BPE নতুন token তৈরি করে **merge rule** শিখে।

Merge rule মানে:

vocabulary-র মধ্যে যে দুইটা **token** পাশাপাশি সবচেয়ে বেশি বার আসে, সেগুলো একসাথে জুড়ে নতুন token বানানো।

শুরুতে merge করলে ২ অক্ষরের token হবে, পরে ৩ অক্ষরের, তারপর আরও বড় subword হবে।

৩) উদাহরণ দিয়ে বুঝি

ধরি, corpus-এ শব্দগুলোর frequency হলো:

("hug", 10), ("pug", 5), ("pun", 12), ("bun", 4), ("hugs", 5)

মানে:

- "hug" আছে 10 বার
- "pug" আছে 5 বার
- "pun" আছে 12 বার
- "bun" আছে 4 বার
- "hugs" আছে 5 বার

প্রথম ধাপ: শব্দগুলোকে অক্ষরে ভাঙা

শুরুতে প্রতিটি word-কে character-এ split করা হবে:

("h" "u" "g", 10)
("p" "u" "g", 5)
("p" "u" "n", 12)
("b" "u" "n", 4)
("h" "u" "g" "s", 5)

দ্বিতীয় ধাপ: সবচেয়ে বেশি ব্যবহৃত **pair** খোঁজা

এখন দেখি কোন দুইটা পাশাপাশি **token** সবচেয়ে বেশি বার এসেছে।

Pair: ("h", "u")

এটা আছে:

- hug → 10 বার
- hugs → 5 বার

মোট = 15 বার

Pair: ("u", "g")

এটা আছে:

- hug → 10 বার
- pug → 5 বার
- hugs → 5 বার

মোট = 20 বার

এটাই সবচেয়ে frequent pair।

তাই প্রথম **merge rule** হবে:

("u", "g") → "ug"

মানে u আর g মিলে নতুন token হবে ug

এখন vocabulary হবে:

["b", "g", "h", "n", "p", "s", "u", "ug"]

আর corpus হবে:

("h" "ug", 10)

("p" "ug", 5)

("p" "u" "n", 12)

("b" "u" "n", 4)

("h" "ug" "s", 5)

তৃতীয় ধাপ: আবার সবচেয়ে **frequent pair** খোঁজা

এখন pair দেখি।

Pair: ("u", "n")

এটা আছে:

- pun → 12 বার
- bun → 4 বার

মোট = 16 বার

Pair: ("h", "ug")

এটা আছে:

- hug → 10 বার
- hugs → 5 বার

মোট = 15 বার

তাই এবার সবচেয়ে frequent হলো:

("u", "n") -> "un"

এখন vocabulary হবে:

["b", "g", "h", "n", "p", "s", "u", "ug", "un"]

আর corpus হবে:

("h" "ug", 10)

("p" "ug", 5)

("p" "un", 12)

("b" "un", 4)

("h" "ug" "s", 5)

চতুর্থ ধাপ: আবার merge

এখন সবচেয়ে frequent pair হলো:

("h", "ug")

কারণ এটা আছে:

- hug → 10 বার
- hugs → 5 বার

মোট = 15 বার

তাই merge rule হবে:

("h", "ug") -> "hug"

এখন vocabulary হবে:

["b", "g", "h", "n", "p", "s", "u", "ug", "un", "hug"]

আর corpus হবে:

("hug", 10)

("p" "ug", 5)

("p" "un", 12)

("b" "un", 4)

("hug" "s", 5)

এরপর কী হবে?

এভাবেই merge চলতে থাকবে যতক্ষণ না desired vocabulary size পূর্ণ হয়।

এখানে পরের **merge rule** কী হতে পারে?

সবচেয়ে সম্ভবত হবে:

("p", "un") -> "pun"

কারণ ("p", "un") pair মোট 12 বার এসেছে।

8) Tokenization Algorithm — নতুন শব্দ tokenize কীভাবে হয়?

Training-এর পরে যখন নতুন কোনো input আসে, তখন BPE এই ধাপগুলো অনুসরণ করে:

1. **Normalization**
টেক্সটকে standard form-এ আনা
2. **Pre-tokenization**
আগে থেকে কিছু basic split করা
3. শব্দকে **character**-এ ভাঙা
4. **Training**-এ শেখা **merge rule** গুলো একে একে **apply** করা

শেখা **merge rules** ছিল:

("u", "g") -> "ug"

("u", "n") -> "un"

("h", "ug") -> "hug"

উদাহরণ ১: **"bug"**

শুরুতে split হবে:

"b" "u" "g"

তারপর:

- ("u", "g") → ug

তাই final token হবে:

["b", "ug"]

উদাহরণ ২: **"mug"**

শুরুতে split:

"m" "u" "g"

কিন্তু **m** base vocabulary-তে ছিল না।

তাই **m** হবে [UNK]

আর **u g** merge হয়ে **ug** হবে।

Final:

["[UNK]", "ug"]

উদাহরণ ৩: **"thug"**

শুরুতে split:

"t" "h" "u" "g"

এখানে **t** vocabulary-তে নেই, তাই:

"[UNK]" "h" "u" "g"

তারপর:

- ("u", "g") → ug
- ("h", "ug") → hug

Final token হবে:

["[UNK]", "hug"]

কেন **BPE** দরকার?

BPE দরকার কারণ:

- সব শব্দ vocabulary-তে রাখা সম্ভব না
- নতুন শব্দ প্রায়ই আসে
- subword-based tokenization model-কে flexible করে
- rare word বা unknown word handle করা সহজ হয়
- memory ও efficiency ভালো হয়